

Cassandra Web Sanitizer - Report

Programmazione di Sistema

Introduzione

Cassandra Web Sanitizer è un tool di sicurezza difensivo scritto in Rust. Funge da livello di sanitizzazione intermedio intercettando pagine web non attendibili tramite link, strutture di file locali e risorse scaricate dalla rete, per poi neutralizzare o riscrivere potenziali elementi pericolosi prima che possano causare danni all'utente.

Cassandra è progettato come un sistema modulare composto da due componenti:

1. Una libreria riutilizzabile (`sanitizer_engine`) che implementa il motore di scansione, i crawler di rete ricorsivi, il rilevamento dei file tramite magic-number (sniffing), i riscrittori di flusso (stream rewriter), i parser di documenti...
2. Un'applicazione da riga di comando (`cli_application`) che permette di utilizzare la libreria di sanitizzazione attraverso un'interfaccia grafica.

La progettazione si affida interamente a **safe Rust**, impiegando una tecnica di **token streaming zero-copy** (utilizzando `lol_html`) al fine di evitare il sovraccarico prestazionale e le vulnerabilità di sicurezza legate alla materializzazione di interi alberi DOM in memoria. Per valutarne l'efficacia, Cassandra è stato testato rispetto a una suite di test contenente sia pagine benigne che risorse pericolose (XSS, XML entity bomb, elementi PDF attivi, confusione omografa IDN, SSRF e MIME confusion). La valutazione sperimentale mostra che Cassandra ottiene un **tasso di rilevamento del 100%** sul corpus malevolo, mantenendo un footprint di memoria di picco estremamente ridotto pari a **49,80 MB** e dimostrando una buona scalabilità su CPU parallele, limitata tuttavia dalla fase sequenziale di scrittura del log su disco.

1. Analisi dei Requisiti

1.1 Requisiti Funzionali

L'architettura funzionale di Cassandra è guidata dalla necessità di difendere il sistema dell'utente. Il sistema neutralizza sei principali minacce:

1. Sanitizzazione dell'HTML:

- **Gestori di Eventi Inline:** Rimozione degli attributi di evento (es. `onclick` , `onerror` , `onload` , `onmouseover` , `onfocus`) da tutti i tag HTML.
- **Blocchi Script:** Neutralizzazione o rimozione dei tag `<script>` a meno che il loro hash di contenuto o il dominio sorgente non corrispondano esplicitamente a un elemento in whitelist.
- **Protocolli Pericolosi:** Intercettazione e sanitizzazione di URI `javascript:` e `data:` presenti negli attributi `href` e `src` .

- **Restrizioni sulle Origini:** Verifica dei tag `<iframe src="...">` e `<object data="...">`, rimuovendoli o bloccandoli se puntano a domini esterni all'elenco delle origini approvate.
- **Reindirizzamenti Meta:** Rimozione completa dei tag `<meta http-equiv="refresh">` per bloccare attacchi di auto-redirezione lato client.
- **Estrazione Estesa dei Tag:** Analisi e sanitizzazione dei collegamenti e delle risorse all'interno di elementi nidificati, inclusi `<form>`, `<audio>`, `<video>`, `<embed>`, `<track>`, `<area>` e `<input>`.

2. MIME Confusion e Content Sniffing:

- Convalida proattiva delle risorse ignorando l'intestazione `Content-Type` fornita dal server HTTP remoto. Il motore esamina i primi byte (magic numbers) dei file per determinare il loro tipo effettivo (HTML, CSS, JS, JPEG, PNG, PDF) e rifiuta i file con intestazioni non corrispondenti.

3. Attacchi Omografi Unicode:

- Rilevamento e prevenzione del domain spoofing mediante la verifica degli URL estratti. Il componente del dominio viene analizzato, normalizzato e controllato alla ricerca di pattern di omografi IDN (Internationalized Domain Name), neutralizzando i collegamenti che usano glifi visivamente simili.

4. Scansione dei Documenti con Contenuto Attivo:

- Ispezione dei documenti PDF per individuare contenuti attivi eseguibili integrati. Esegue una scansione della struttura binaria del PDF alla ricerca dei dizionari `/JavaScript`, `/JS` o `/OpenAction` che attivano comportamenti dinamici all'interno dei lettori PDF, neutralizzando il file in caso di rilevamento.

5. Protezione da attacchi Denial of Service (DoS):

- **Compression Bomb:** Limitazione dei rapporti di decompressione e della dimensione massima dei byte decompressi per le risorse web compresse (es. codifiche `gzip` o `deflate`).
- **Espansione di Entità XML/HTML:** Blocco delle dichiarazioni di entità XML personalizzate (es. i riferimenti ricorsivi di tipo `<!ENTITY lol "lol">`) per difendersi dagli attacchi DoS di espansione.
- **Dimensioni delle Immagini ed Esaurimento Risorse:** Filtro su asset di dimensioni eccessive o risorse potenzialmente infinite.

6. Server-Side Request Forgery (SSRF):

- Durante il recupero delle risorse remote, blocco dell'accesso a indirizzi di loopback (`127.0.0.0/8`, `::1`), subnet private (`10.0.0.0/8`, `192.168.0.0/16`, `172.16.0.0/12`), spazi link-local, indirizzi multicast e intervalli CGNAT (Carrier-Grade NAT) per evitare che Cassandra venga usato per effettuare scansioni di reti interne.

Reportistica Strutturata e Configurazione CLI:

Cassandra scrive un file consolidato `cassandra.log` e un singolo report strutturato `report.json` in cui viene verificata ogni azione (regole attivate, offset dei byte, valori

originali e modifiche) per ciascun input.

Supporta inoltre configurazioni della CLI tramite file di policy TOML personalizzati, esecuzione batch, impostazione del numero di thread worker e codice di uscita non nullo se le regole di blocco della policy risultano in un rifiuto del contenuto.

1.2 Requisiti Non Funzionali

- **Sicurezza della Memoria:** Assenza totale di vulnerabilità legate alla gestione della memoria (es. buffer overflow, use-after-free) garantendo che i livelli di parsing e riscrittura siano scritti in safe Rust.
 - **Footprint di Memoria Minimo:** Elaborazione di file di grandi dimensioni e compiti batch senza caricare i documenti interi in memoria, implementando tokenizer di flusso per limitare l'uso della RAM.
 - **Concorrenza e Scalabilità dei Thread:** Distribuzione dei compiti di elaborazione sui core della CPU disponibili, utilizzando scheduler asincroni per gestire in parallelo letture del disco e I/O di rete.
 - **Budget di Tempo e Risorse:** Applicazione di limiti superiori rigidi sulla profondità di recupero delle risorse (`max_depth`), sul numero totale di richieste HTTP (`max_requests`) e sui byte totali scaricati (`max_bytes`) per garantire che l'elaborazione termini sempre in modo sicuro anche sotto attacco.
-

2. Decisioni Architettureali

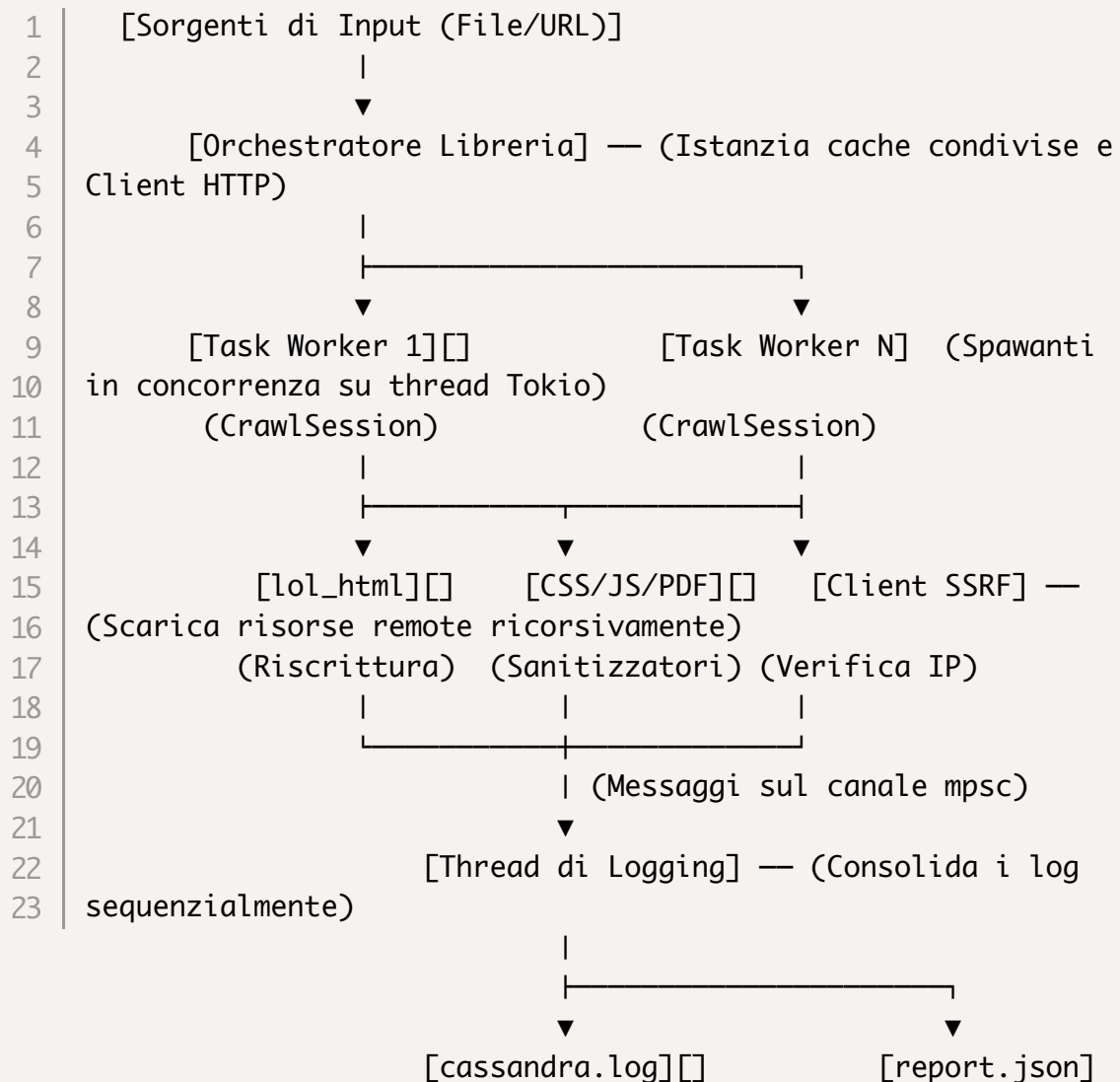
Cassandra è organizzato come un workspace Cargo multi-crate per separare nettamente la logica del motore di sanitizzazione dall'interfaccia utente della CLI.

```

1 | cassandra/ (Workspace Root)
2 |   └─ Cargo.toml
3 |   └─ policies/
4 |       └─ default.toml      <-- Policy Predefinita
5 |   └─ sanitizer_engine/    <-- Crate Libreria Core
6 | (cassandra)
7 |   └─ Cargo.toml
8 |   └─ src/
9 |       └─ lib.rs           <-- Orchestratore della Libreria
10 |       └─ crawl_session.rs <-- Crawler
11 |       └─ http_client.rs   <-- Client HTTP Safe Anti-SSRF
12 |       └─ html.rs          <-- Riscrittore HTML in
13 | streaming (lol_html)
14 |   └─ log.rs              <-- Thread di Logging su Canale
15 | MPSC
16 |   └─ errors.rs           <-- Errori Personalizzati
17 |   └─ policy.rs           <-- Definizione Policy TOML
18 |   └─ rules.rs            <-- Definizioni delle regole
19 |   └─ url.rs              <-- Verifica URL e IDN
20 |   └─ resources/         <-- Sanitizzatori Specifici per
21 | Risorse
22 |   └─ mod.rs              <-- Inizializzatore Modulo e
23 | Gestore Nomi File
24 |   └─ css.rs              <-- Sanitizzatore CSS
25 |   └─ entities.rs         <-- Scanner Entità XML (Entity
26 | Bomb)
27 |   └─ images.rs           <-- Rimozione Metadati e EXIF
28 | Immagini
29 |   └─ javascript.rs       <-- Scanner Parole Chiave JS
30 |   └─ mime.rs              <-- Rilevatore Magic Number
   | (Sniffer)
   |   └─ pdf.rs            <-- Scanner Contenuti Attivi PDF
   |   └─ cli_application/  <-- Crate Frontend CLI
   | (cli_application)
   |   └─ Cargo.toml
   |   └─ src/
   |       └─ main.rs        <-- Gestore ed Esecutore dei
   | Comandi CLI
   |   └─ bin/
   |       └─ evaluation_runner.rs <-- Runner della
   Valutazione Sperimentale

```

2.1 Pipeline di Elaborazione Dati



- **L'Orchestrator** (`lib.rs`): Inizializza una mappa globale thread-safe delle URL visitate (`url_map` protetta da `Arc<Mutex<HashMap<Url, usize>>>`) e istanzia il client HTTP safe. Itera sulle sorgenti di input, costruisce le istanze di `CrawlSession` e le pianifica sull'esecutore asincrono di Tokio.
- **La Crawl Session** (`crawl_session.rs`): Gestisce l'elaborazione dei singoli file e il crawling delle URL. Tiene traccia delle limitazioni delle risorse (`total_requests` , `total_bytes`) per sessione utilizzando contatori locali protetti da mutex e genera asincronamente nuove richieste per le risorse identificate (es. fogli di stile, script) fino alla profondità massima `max_depth` .
- **Il Client HTTP Safe** (`http_client.rs`): Intercetta le richieste in uscita, risolve i nomi DNS e filtra i blocchi IP privati, loopback e CGNAT. Limita la comunicazione alle connessioni HTTPS e analizza i magic numbers dei corpi delle risposte.
- **Parser HTML in Flusso** (`html.rs`): Integra `lol_html` . Legge i file HTML in piccoli chunk, applicando riscrittori basati su selettori CSS per intercettare

elementi critici (es. `<script>`, `<iframe>`, `<a>`) e modificarne le proprietà al volo senza allocare memoria superflua.

- **Il Logger Consolidato** (`log.rs`): I worker inviano log di avanzamento ed errori di regola (`RuleError`) tramite un canale `mpsc` . Un thread in background dedicato legge dal canale, stampando il progresso sulla riga di comando. Genera poi il file `cassandra.log` e produce il report strutturato finale `report.json` .

NOTA Impatto sul consumo di RAM

I dati accumulati in memoria per il log e il report sono strutturalmente molto leggeri:

- Una riga di log media occupa circa un centinaio di byte.
- Anche nel nostro scenario più pesante (carico grande da ****7000 file**** con circa 15.000 righe di log e report), l'occupazione totale in RAM delle stringhe e dei vettori accumulati è di circa ****1.5 - 2.5 MB****.
- Poiché il consumo di RAM di picco misurato è di circa ****50 MB**** (occupati principalmente da runtime Tokio, stack dei thread, buffer di connessione socket e strutture dati globali), liberare 2 MB di RAM non darebbe alcun beneficio tangibile al sistema.

2.2 API Pubbliche del Crate

Il crate di libreria (`sanitizer_engine`) espone un'interfaccia pubblica che permette l'integrazione di Cassandra come dipendenza all'interno di altre applicazioni Rust. Le API principali fornite sono:

- **InputSource** : Enum che rappresenta la sorgente da scansionare, supportando file locali o URL remoti:

```
rust
```

```
pub enum InputSource {  
  
    File(PathBuf),  
  
    Url(Url),  
  
}
```

- **library(...)** : La funzione di ingresso principale per l'esecuzione della pipeline di sanitizzazione. Accetta un riferimento al runtime di Tokio, una lista di sorgenti, la policy di sicurezza, il percorso di output e il canale MPSC per la trasmissione asincrona dei messaggi di log:

```
rust
```

```
pub fn library(  
  
    runtime: &Runtime,  
  
    sources: Vec<InputSource>,
```

```

    policy: Arc<Policy>,

    output_dir: Arc<PathBuf>,

    tx: Sender<LoggerMessage>,

) -> Result<(), SanitizerError>;

```

- `logging_thread(...)` : Funzione pubblica esposta dal modulo `log` per avviare il ciclo di consumo sincrono dei messaggi dal canale MPSC, incaricata di generare i file consolidati `cassandra.log` e `report.json` :

```

rust

pub fn logging_thread(

    output: &Path,

    console_level: LogLevel,

    file_level: LogLevel,

    sources: &[InputSource],

    max_subresources: usize,

    channel: Receiver<LoggerMessage>,

) -> (bool, f64, f64);

```

- `Policy` : La struttura di configurazione che definisce le regole di sanitizzazione (esprese in TOML), suddivisa in sotto-policy: `HtmlPolicy` , `UrlsPolicy` , `ResourcesPolicy` e `ConnectionsPolicy` .
- `RuleWithValue<T>` e `ReplaceRule<T>` : Esposte nel modulo `rules` , permettono di associare livelli di log personalizzati alle limitazioni numeriche (es. `MaxBytes` o `MaxSubresources`) e alle azioni di sostituzione.
- **Sanitizzatori Specifici (`resources/`)**: La libreria espone funzioni di utilità riutilizzabili per manipolazioni mirate, tra cui `strip_jpeg_metadata` / `strip_png_metadata` per la rimozione EXIF, ed `EntityScanner` per il rilevamento incrementale di entità XML sospette.

3. Aspetti di System Programming in Rust

3.1 Modello di Concorrenza e Primitive di Sincronizzazione

Cassandra si affida al **runtime multi-thread di Tokio** per eseguire in parallelo le operazioni CPU-bound ed I/O-bound. Per coordinare i dati tra i thread in totale sicurezza, il motore utilizza le seguenti primitive di concorrenza Rust:

- `Arc<Mutex<HashMap<Url, usize>>>` : Utilizzato per la cache globale delle URL visitate. I worker controllano questa mappa per evitare download duplicati. Poiché le ricerche sono immediate, il lock viene mantenuto per pochissimo tempo, riducendo al minimo la contesa. Si è preferito `parking_lot::Mutex` rispetto a `std::sync::Mutex` per via del suo comportamento di spin-lock più veloce sui percorsi ad alta contesa.
- `Arc<Policy>` : Le policy dichiarative sono in sola lettura durante l'esecuzione. Avvolgere la policy in un `Arc` consente ai worker di fare riferimento alle regole contemporaneamente senza il sovraccarico di un lock.
- `std::sync::mpsc::channel` : Si è scelto un canale asincrono di passaggio messaggi per separare l'elaborazione dalla scrittura fisica su disco. Se i worker scrivessero contemporaneamente sul file di log, rimarrebbero bloccati a causa dei lock del filesystem. Invece, i worker inviano i log al canale e un unico thread in background gestisce la scrittura in modo sequenziale.
- **Tokio Runtime Handles**: I thread worker ricevono una copia di `tokio::runtime::Handle` per avviare ricorsivamente compiti di crawling di sottorisorse senza dover passare l'intero oggetto runtime, mantenendo l'orchestrazione dei task leggera.

```
1 // crawl_session.rs - Spawning asincrono ricorsivo delle
2 sottorisorse
3 let clone = Arc::clone(self);
4 self.rt_handle.spawn(async move {
5     if let Err(e) = clone
6         .crawl_subresource(url, local_name, depth, &logger)
7         .await
8     {
9         logger.error(e);
10    }
11 });
```

3.2 Lifetime, Ownership e Parsing Zero-Copy

Gli strumenti di sicurezza devono elaborare flussi di byte non attendibili senza introdurre colli di bottiglia prestazionali. Cassandra ottiene un throughput elevato combinando l'approccio zero-copy con copie di memoria altamente ottimizzate:

- **Tokenizzazione in Flusso**: Utilizzando `lol_html`, il motore elabora i file HTML all'interno di buffer di finestra di 8KB. Analizza, trasforma e scrive i byte direttamente nel flusso del file di output senza generare un DOM in memoria. L'occupazione di memoria rimane costante indipendentemente dalla dimensione del file.
- **Zero-Copy vs Copie Ottimizzate**: Il parsing HTML e la validazione dei percorsi

stringa sono strettamente *zero-copy*. Le sottorisorse binarie o complesse (immagini JPEG/PNG, fogli CSS e PDF) vengono invece bufferizzate interamente in memoria per consentire l'accesso casuale e l'analisi strutturale (es. verifica dei chunk PNG o dei dizionari PDF). La riscrittura di queste risorse non è strettamente zero-copy poiché alloca un nuovo buffer per i byte purificati (es. escludendo i segmenti EXIF); tuttavia, l'algoritmo evita qualsiasi deserializzazione intermedia pesante (come la decodifica dei pixel o la materializzazione di un albero sintattico PDF), limitandosi a copiare selettivamente e linearmente i segmenti sicuri del buffer originale.

- **Borrowing vs Allocating:** Dove possibile, le strutture dati prendono in prestito slice di stringhe (`&str`) dai buffer di origine anziché allocare stringhe proprietarie (`String`) sull'heap. I lifetime (es. `'a`) vincolano le regole alla durata del buffer di input, garantendo che la memoria venga liberata non appena il parser avanza.
- **Copy-on-Write (`Cow<'a, str>`):** Utilizziamo `Cow` durante la corrispondenza dei pattern. Se una stringa è pulita, restituiamo un riferimento preso in prestito (`Cow::Borrowed(&str)`); se la stringa richiede una normalizzazione (es. rimozione di spazi bianchi o normalizzazione Unicode), allochiamo memoria e restituiamo una copia proprietaria (`Cow::Owned(String)`), risparmiando cicli CPU sui testi sicuri.

3.3 Gestione degli Errori Type-Safe e Mappatura delle Azioni di Policy

La gestione degli errori in Cassandra è progettata per differenziare nettamente i guasti di sistema (es. fallimento della scrittura di un file) dai blocchi imposti dalle policy di sicurezza (es. blocco di uno script).

- **Enum Type-Safe:** Le strutture degli errori personalizzate sono definite utilizzando la libreria `thiserror` :

```
1  #[derive(Debug, Clone, Serialize, Deserialize)]
2  #[serde(tag = "type", rename_all = "snake_case")]
3  pub enum RuleError {
4      MimeMismatch { expected: Option<String>, actual:
5  Option<String> },
6      BlockedMetaRefresh { original: String },
7      ActiveContent { file: String, content: String },
8      Replace {
9          inner: RuleReplaceError,
10         replacement: Option<String>,
11         offset: Range<usize>,
12     },
13     // ...
}
```

- **Mappatura delle Azioni di Policy:** Con la convalida di una regola fallisce, viene generato un `RuleError`. Il motore esamina l'azione configurata nella policy (Ignore, Warn, Replace, Error/Deny) ed esegue la mappatura corrispondente:
 - `Ignore`: Ignora l'errore procedendo normalmente.
 - `Warn`: Invia un log di avviso sul canale e mantiene il contenuto originale.
 - `Replace`: Sostituisce l'attributo con un valore segnaposto sicuro (es. `src="blocked:url"`) e registra la modifica.
 - `Error`: Rimuove l'attributo o il tag intero, registra il blocco e incrementa il contatore di errori complessivo, portando la CLI a restituire un codice di uscita non zero alla chiusura.

```
1 // log.rs - Convalida degli errori e gestione delle azioni di
2 policy
3 pub fn handle<T: Into<SanitizerMessage>>(self, logger: &impl
4 Log, message: T) -> Result<(), T> {
5     if self == LogLevel::Error {
6         Err(message)
7     } else {
8         logger.log(self, message);
9         Ok(())
10    }
11 }
```

3.4 Analisi del Codice Unsafe

Cassandra è scritto interamente in safe Rust per ridurre al minimo i bug di memoria.

4. Valutazione Sperimentale

La valutazione sperimentale è stata condotta su un sistema macOS eseguendo `cli_application/src/bin/evaluation_runner.rs` ed elaborando i grafici tramite `plot_results.py`.

4.1 Correttezza

La correttezza del motore di sanitizzazione è stata valutata rispetto a una serie di dati reali rappresentanti una vasta gamma di minacce: cross-site scripting (XSS), XML bomb, IDN, richieste di rete SSRF, bypass di fogli di stile CSS ed elementi binari attivi (PDF con codice JavaScript).

Riepilogo delle Metriche

- **True Positives (TP):** 11

- True Negatives (TN): 2
- False Positives (FP): 1
- False Negatives (FN): 0
- Tasso di Rilevamento Globale (Sensibilità): 100,00%
- Tasso di Falsi Positivi: 33,33%

Risultati Dettagliati

File	Verdetto Atteso	Verdetto Effettivo	Regole Attese	Regole Effettive	Stato
malicious/untrusted_origin.html	malicious	malicious	dangerous_domain, dangerous_origins	dangerous_domain, dangerous_origins	MATCH (TP)
malicious/unicode_confusion.html	malicious	malicious	idn	idn	MATCH (TP)
malicious/xml_bomb.html	malicious	malicious	xml_entity_declaration	xml_entity_declaration	MATCH (TP)
malicious/ssrf_attempt.html	malicious	malicious	dangerous_scripts	dangerous_scripts	MATCH (TP)
malicious/malicious_urls.html	malicious	malicious	dangerous_domain, idn	dangerous_domain, idn	MATCH (TP)
benign/clean_doc.pdf	benign	benign	none	none	MATCH (TN)
malicious/idn_only.html	malicious	malicious	idn	idn	MATCH (TP)
benign/safe.html	benign	benign	none	none	MATCH (TN)
malicious/pdf_js_bomb.pdf	malicious	malicious	active_content	active_content	MATCH (TP)
benign/crawler_test.html	benign	malicious	none	dangerous_scripts	MISMATCH (FP)

st.html					
malicious/dangerous_styles.css	malicious	malicious	dangerous_css	dangerous_css	MATCH (TP)
malicious/xss.html	malicious	malicious	event_handlers, dangerous_scripts	dangerous_scripts, event_handlers	MATCH (TP)
malicious/broadened_urls.html	malicious	malicious	dangerous_domain	dangerous_domain	MATCH (TP)
malicious/dangerous_script.js	malicious	malicious	dangerous_js	dangerous_js	MATCH (TP)

Analisi dei Falsi Positivi: Il file `benign/crawler_test.html` ha attivato la regola `dangerous_scripts` in quanto conteneva uno script inline non presente nella whitelist predefinita. Poiché il parser è configurato in modalità strettamente difensiva, qualsiasi blocco di script non espressamente approvato viene categorizzato come non sicuro e sanitizzato, producendo un falso positivo sui siti che usano codice inline senza aver registrato gli opportuni hash.

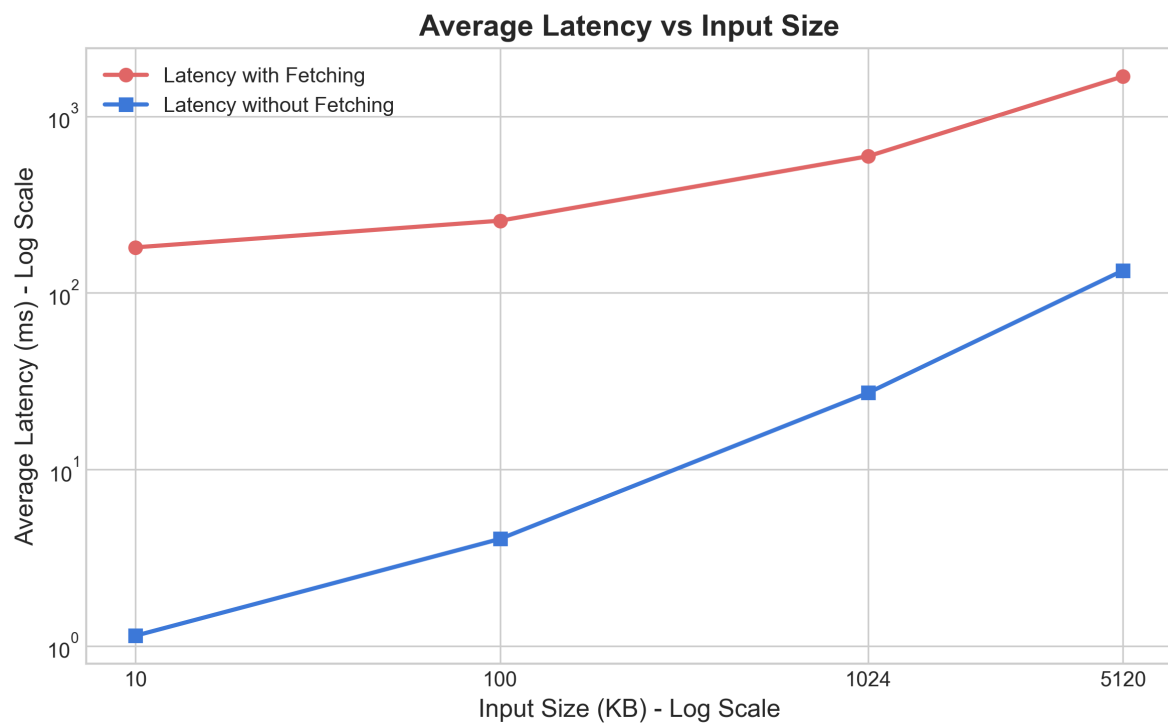
4.2 Performance (Latenza vs Dimensione dell'Input)

Il throughput e la latenza sono stati misurati utilizzando file HTML sicuri con dimensioni variabili da 10KB a 5MB, abilitando e disabilitando il caricamento delle risorse remote collegate.

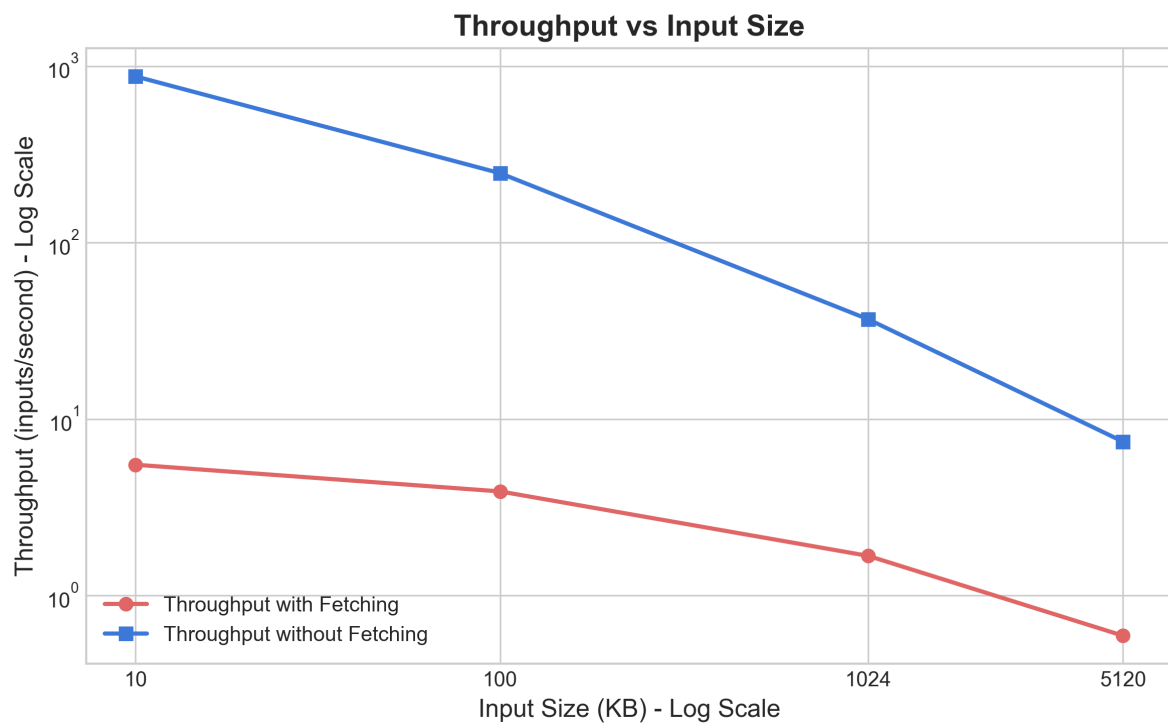
Le tabelle seguenti misurano la latenza per-input in *ms*, e il throughput in *inputs per second*.

Dimensione	Latenza (No Fetch)	Throughput (No Fetch)	Latenza (Con Fetch)	Throughput (Con Fetch)
10 KB	1.14 ms	875.71 ips	181.22 ms	5.52 ips
100 KB	4.04 ms	247.65 ips	256.65 ms	3.90 ips
1 MB	27.18 ms	36.80 ips	595.23 ms	1.68 ips
5 MB	134.10 ms	7.46 ips	1686.80 ms	0.59 ips

Latenza rispetto alla dimensione dell'input



Throughput rispetto alla dimensione dell'input



Osservazioni:

- **Andamento Asintotico Lineare ($O(N)$) e Costi Fissi:** Quando opera esclusivamente come parser locale (senza scaricare le risorse remote), la latenza non è perfettamente lineare su file estremamente piccoli, ma si stabilizza verso un andamento lineare puro man mano che la dimensione del file cresce. Per file piccoli (es. da 10KB a 100KB), la latenza passa da **1,14 ms** a **4,04 ms** (crescita di 3,5x a fronte di un aumento di 10x della dimensione) a causa dell'overhead fisso di inizializzazione del motore di riscrittura, allocazione dei buffer di output e setup dei descrittori. Tuttavia, per file grandi questo overhead viene ammortizzato: passando da 1MB (**27,18 ms**) a 5MB (**134,10 ms**), la latenza aumenta di **4,93x** a fronte di un incremento di dimensione esatto di **5x**. La velocità di elaborazione si assesta stabilmente a circa 0,026 ms per KB (~38 MB/s), confermando l'andamento lineare asintotico $O(N)$ e l'efficacia del parsing zero-copy di `lol_html`.
- **Mitigazione della Concorrenza sul Subfetching (Crescita Sub-lineare):** Quando il subfetching delle sottorisorse è abilitato, la latenza cresce in modo decisamente meno che proporzionale (sub-lineare) rispetto al numero totale di risorse da scaricare. Ad esempio, passando da 10KB (2 sottorisorse) a 5MB (60 sottorisorse), il numero totale di risorse da scaricare aumenta di **30 volte**, ma la latenza passa da **181 ms** a **1686 ms** (crescita di sole **9,3 volte**). Questo andamento evidenzia l'efficacia del **modello concorrente basato su Tokio**: le richieste HTTP in uscita vengono avviate asincronamente in parallelo anziché sequenzialmente. Di conseguenza, il tempo totale è governato principalmente dal tempo di round-trip (RTT) delle connessioni parallele e non dalla somma sequenziale dei singoli download. Solo al raggiungimento di 60 risorse concorrenti (a 5MB), i limiti del pool di connessioni, la contesa sulla cache URL e il costo locale di CPU-parsing iniziano a produrre una crescita più marcata.

4.3 Scalability ed Efficienza della Pipeline Parallela

La scalabilità è stata misurata elaborando due diversi tipi di carico di lavoro al variare del numero di thread worker (1, 2, 4, 8 e 16):

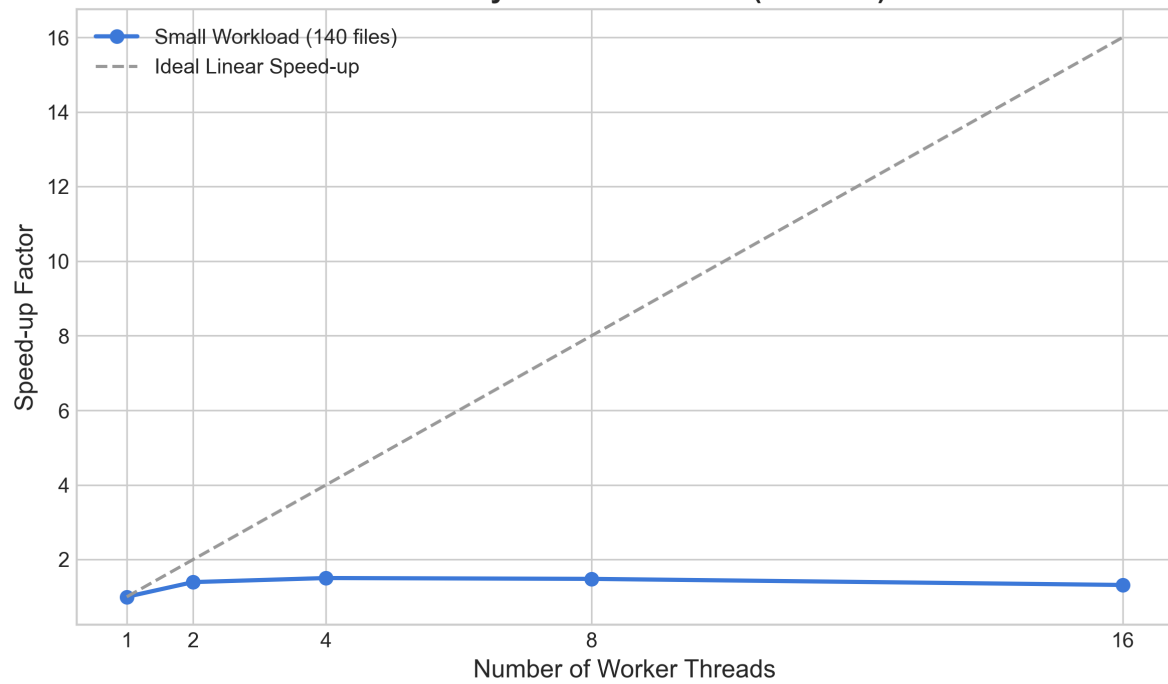
- **Carico Piccolo (140 file):** Un batch ad esecuzione rapida che si completa in circa ~60ms.
- **Carico Grande (7000 file):** Un batch di grandi dimensioni che richiede circa ~3 secondi di elaborazione.

Prestazioni di Scalabilità (Durate di Esecuzione)

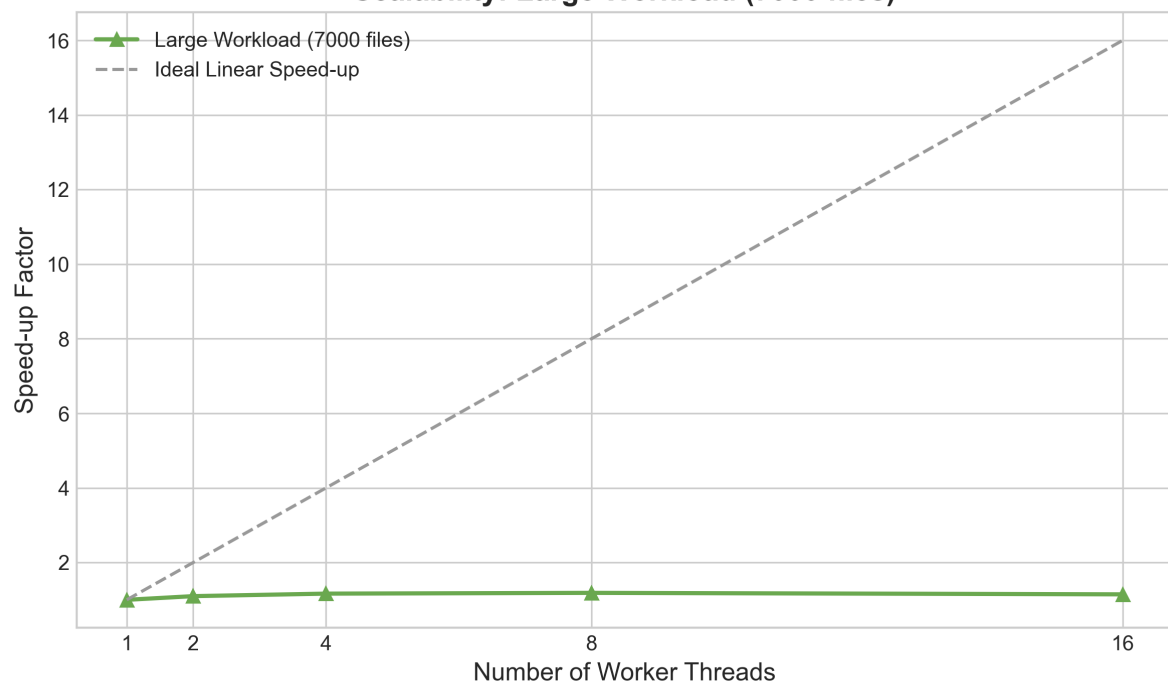
Numero Thread	Durata Carico Piccolo	Speedup Carico Piccolo	Durata Carico Grande	Speedup Carico Grande
1	0.064 s	1.00x	2.456 s	1.00x
2	0.046 s	1.39x	2.236 s	1.10x
4	0.043 s	1.50x	2.106 s	1.17x
8	0.043 s	1.48x	2.070 s	1.19x
16	0.049 s	1.32x	2.141 s	1.15x

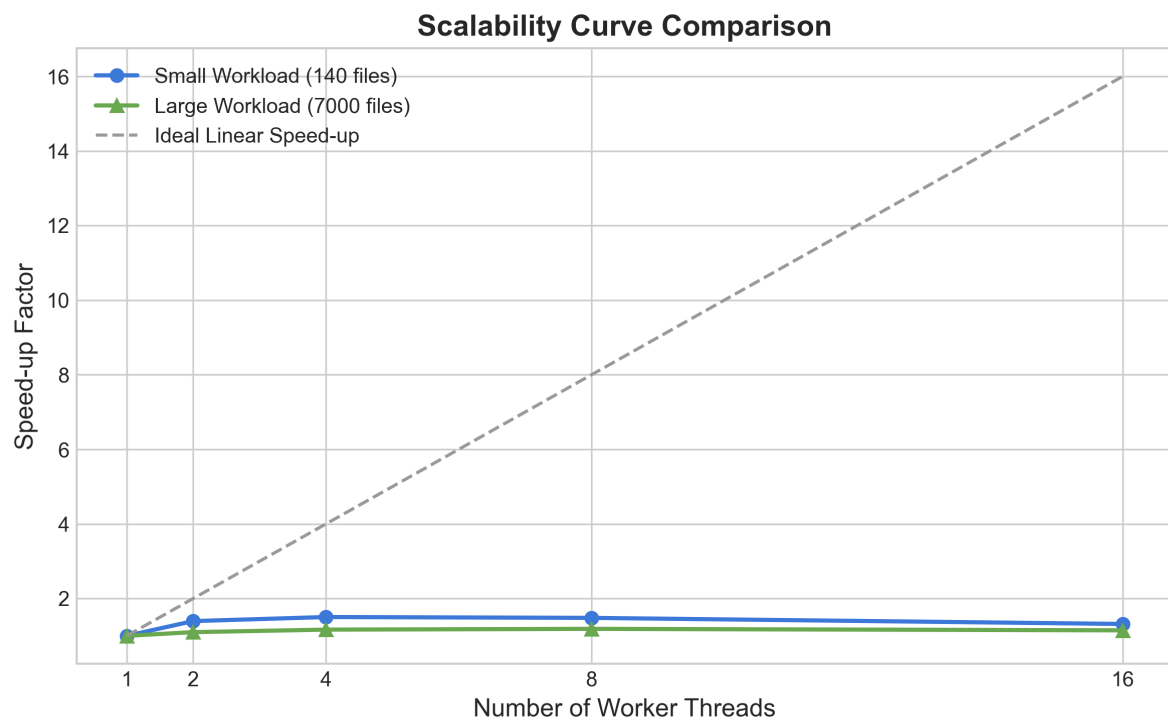
Curve di Speedup

Scalability: Small Workload (140 files)



Scalability: Large Workload (7000 files)

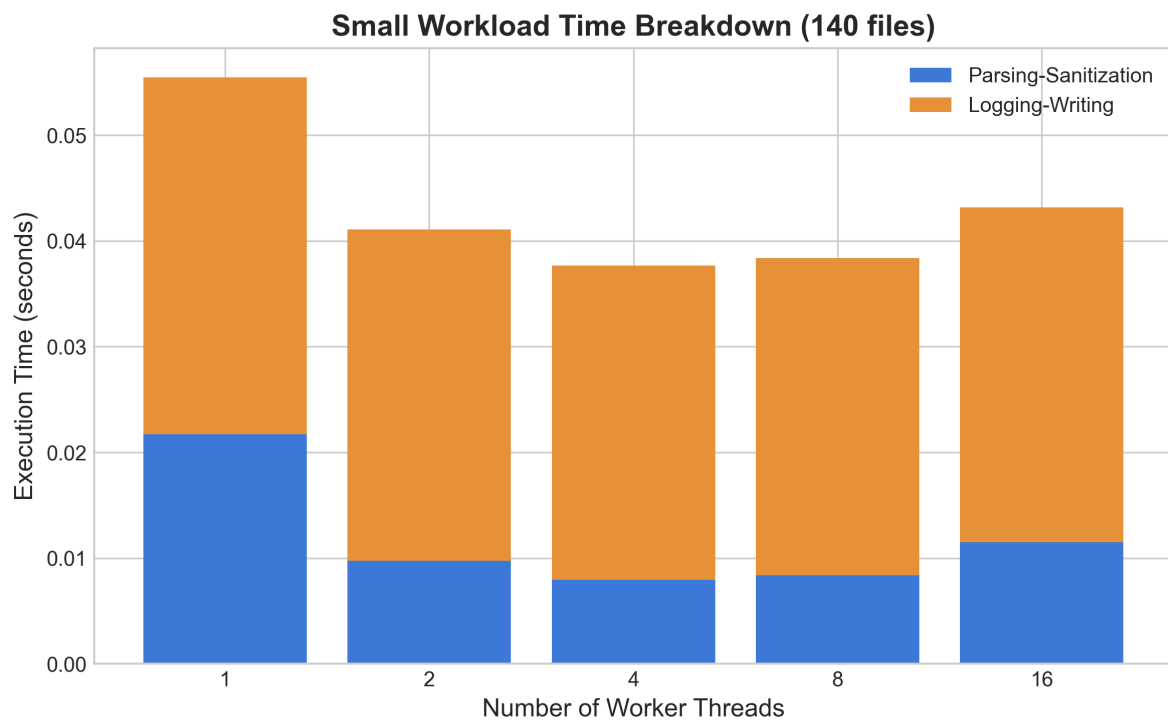




Suddivisione dei Tempi delle Fasi (Parsing vs Scrittura/Logging)

Numero Thread	Parse Piccolo	Scrittura Piccolo	Totale Piccolo	Parse Grande	Scrittura Grande	Totale Grande
1	0.022 s	0.034 s	0.056 s	0.785 s	1.407 s	2.192 s
2	0.010 s	0.031 s	0.041 s	0.522 s	1.455 s	1.977 s
4	0.008 s	0.030 s	0.038 s	0.419 s	1.431 s	1.850 s
8	0.008 s	0.030 s	0.038 s	0.378 s	1.436 s	1.814 s
16	0.011 s	0.032 s	0.043 s	0.359 s	1.526 s	1.885 s

Suddivisione dei Tempi di Elaborazione



Osservazioni:

- **Speedup del Parser:** La fase CPU-bound di parsing ed elaborazione si dimostra altamente scalabile all'aumentare dei thread. Nel carico di grandi dimensioni, il tempo del parser si riduce da **0,785s** (con 1 thread) a soli **0,359s** (con 16 thread), garantendo uno **speedup effettivo pari a 2,19x**.

- **Collo di Bottiglia di Scrittura:** La fase di logging e scrittura sul disco (scrittura dei log consolidati e serializzazione del JSON) rimane costante a circa **1,4s - 1,5s** in tutte le esecuzioni. Poiché il thread di logging opera in modo strettamente sequenziale, questa operazione limita la scalabilità massima complessiva dell'intero sistema.
- **Overhead di Scheduling:** Per carichi piccoli, i tempi necessari all'inizializzazione del runtime parallelo e al context switch tra i thread superano i benefici pratici dell'elaborazione concorrente. Troviamo un ottimo nell'esecuzione con soli 4 thread.

4.4 Consumo di Risorse

Durante l'esecuzione della suite di valutazione, l'occupazione massima della RAM è stata pari a:

Picco Resident Set Size (RSS): 49,80 MB

Analisi dell'Efficienza Zero-Copy:

Il ridottissimo consumo di memoria è conseguenza diretta dell'approccio zero-copy:

1. **Token in Flusso:** `lol_html` evita di allocare un DOM in memoria, garantendo un footprint costante.
2. **Riutilizzo dei Buffer:** I buffer vengono riciclati tra i vari stage riducendo le allocazioni sull'heap.
3. **Condivisione tramite Riferimenti:** Elementi pesanti (come i dizionari di domini sospetti) sono condivisi tra i thread tramite riferimenti atomici `Arc<T>` senza alcuna duplicazione di dati.

5. Valutazione Critica

5.1 Limitazioni del Sistema

1. **Analisi Statica Limitata:**
Cassandra si basa esclusivamente su controlli statici delle stringhe e blacklist. Non è in grado di rilevare minacce generate dinamicamente a runtime tramite offuscamenti Javascript complessi (es. uso di `eval` dinamici o concatenazione di stringhe).
2. **Collo di Bottiglia sul Logging Sequenziale:**
La serializzazione del report JSON finale tramite `serde_json::to_writer_pretty` viene eseguita in modalità sincrona e monothread, rappresentando un evidente limite prestazionale sui sistemi con molti core ed elevata concorrenza.
3. **Possibili attacchi DNS Rebind:**
Nonostante il modulo anti-SSRF effettui la risoluzione IP per verificare la destinazione, non memorizza in cache l'indirizzo IP validato per l'intera durata della richiesta di download. Questa assenza potrebbe consentire attacchi DNS

rebinding se l'IP del dominio di destinazione cambia tra il controllo e la connessione reale.

4. **Limiti di Memoria sulle Risorse Scaricate:**

Sebbene `total_bytes` prevenga download infiniti complessivi, i singoli asset remoti vengono completamente scaricati in memoria prima di essere sniffati e sanitizzati. In caso di molteplici download concorrenti ad alta velocità, la memoria heap potrebbe subire una temporanea saturazione.

5.2 Possibili Estensioni Future

1. **Scrittura Asincrona del JSON in Streaming:**

Sostituzione dell'attuale serializzazione finale sincrona con un serializzatore JSON asincrono e in flusso (es. scrittura incrementale degli eventi man mano che avvengono) per rimuovere il collo di bottiglia del thread di log.

2. **Sandboxing Javascript via WebAssembly:**

Integrazione di un interprete Javascript super-leggero all'interno di una sandbox WebAssembly (es. tramite `wasmtime`) per valutare gli script a runtime in sicurezza ed estrapolare minacce offuscate.

3. **Binding Fisico del Socket Anti-SSRF:**

Miglioramento del client HTTP in modo da forzare il socket a connettersi unicamente all'indirizzo IP risolto e convalidato all'inizio, eliminando alla radice gli attacchi di rebind DNS.

4. **Ricaricamento a Caldo delle Policy:**

Utilizzo di librerie di monitoraggio del filesystem (es. il crate `notify`) per aggiornare le definizioni delle regole TOML in tempo reale senza dover interrompere e riavviare Cassandra.